Service Polymorphism: Client-Side Service Binding for Optimizing User-Perceived QoS

Zhengquan Li[1], Long Cheng[2], Myungjin Lee[3], Edith C. H. Ngai[4], and Zheng Song[1]

[1] Department of Computer and Information Science, University of Michigan–Dearborn, United States
{zqli, zhesong}@umich.edu
[2] School of Computing, Clemson University, United States
lcheng2@clemson.edu
[3] Cisco Research, United States
myungjle@cisco.com
[4] The University of Hong Kong, Hong Kong
chngai@eee.hku.hk

Abstract. Even in the AI era, web services remain the dominant approach for accessing remote data and functionalities. When multiple services satisfy the same application requirement, developers typically select and hard-code a single service with the best overall QoS for all end users. However, our empirical measurements show that such one-size-fits-all binding can perform poorly for individual users because web service QoS varies across locations and over time. To address this problem, this paper introduces service polymorphism, a client-side service-binding paradigm that dynamically binds each service invocation to an equivalent service based on QoS observed in the user's context. To maintain fresh QoS knowledge without costly dedicated probing, our solution learns QoS from normal service invocations and uses an exploration–exploitation mechanism to balance invoking QoS-uncertain services for QoS learning and exploiting the currently best-performing service to optimize user-perceived QoS. Furthermore, nearby users collaboratively share QoS observations to reduce redundant exploration overhead. Evaluation results show that our approach reduces user-perceived latency by up to 50%, while remaining lightweight and scalable under increasing workloads.

Keywords: QoS Optimization · Service Binding · Runtime System

1 Introduction

Web services have become the de facto standard for enabling mobile applications (apps) to access remote data and functionalities [17]. Even with the emergence of LLM-based applications, web services remain a primary mechanism for exposing functionalities to AI agents through protocols such as Model Context Protocol (MCP) [16]. Consequently, the Quality of Service (QoS) of web services, including latency, reliability, and availability, plays a critical role in determining the performance and responsiveness of modern applications.

When developing apps, developers often need to choose from different services that satisfy the same functional and non-functional requirements. For example,

a tourism app may use language translation to support communication with local people. Services such as Text Translator [2], NLP Translator [15], and Lecto Translation [14] are "**equivalent services**" because they provide the required translation functionality with acceptable latency, reliability, and accuracy. In current development practice, developers commonly select the service deemed to provide the best overall QoS among its peers and hard-code it into the application for all end users. This assessment is typically based either on QoS claims published by service providers or on aggregated QoS observations [4]. We call such service binding "**one-size-fits-all**."

However, service QoS is not stationary. It varies across locations and time [21] and may drift continuously due to changing network conditions, workloads, and backend deployments. Consequently, a service deemed optimal overall may perform poorly for certain users or runtime contexts. As revealed by our empirical study (Section 2), this issue is widespread among equivalent web services and limits the effectiveness of one-size-fits-all binding.

This paper introduces **service polymorphism**, a software paradigm for optimizing user-perceived QoS. Inspired by the concept of polymorphism in object-oriented programming, where a function or object behaves differently depending on the scenario, a polymorphic service maintains a set of equivalent services, and each service invocation is dynamically bound to the one with the best QoS at invocation time. Fig. 1 illustrates this idea using the translation example: rather than always invoking a hard-coded translation service, service polymorphism allows the app to dynamically switch among equivalent services based on the user's runtime context and observed QoS.

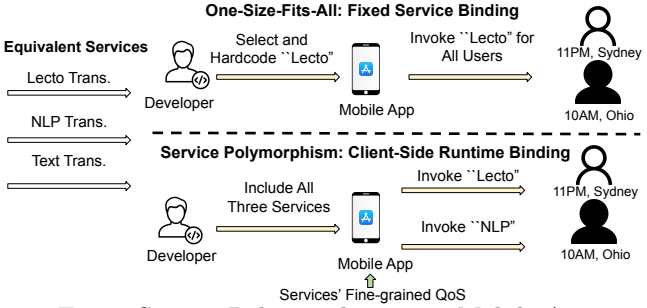


Fig. 1: Service Polymorphism in a Mobile App

The key challenge in enabling service polymorphism is continuously maintaining context-specific QoS knowledge for equivalent services. The accuracy, granularity, and freshness of such QoS knowledge directly affect runtime binding decisions, but collecting it can impose significant overhead on web services and mobile devices. To address this challenge, we design a middleware system for enabling service polymorphism with low-overhead QoS learning. Instead of relying on dedicated QoS probing, the middleware continuously learns service QoS from normal service invocations performed by end users. To balance QoS learning and user-perceived performance, the system models the trade-off between exploring services with uncertain QoS and exploiting the currently best-performing service

as a multi-armed bandit (MAB) problem, solved using a customized upper confidence bound (UCB)-based approach. Furthermore, nearby users collaboratively share QoS observations to reduce redundant exploration overhead and improve QoS estimation quality. To minimize the overhead of QoS sharing, our system leverages pervasive WiFi access points (APs) as lightweight local coordination points, enabling QoS retrieval in less than one millisecond. *Overall, we make the following contributions:*

- We conduct a large-scale study of service QoS from seven global locations, revealing the limitations of one-size-fits-all service binding under location- and time-varying QoS and demonstrating the optimization opportunities provided by equivalent services (Section 2).
- We introduce service polymorphism, a software paradigm that maintains equivalent services and dynamically binds service invocations according to runtime QoS conditions, and implement a middleware that supports fine-grained QoS monitoring (Section 3).
- We build a testbed and multiple apps to evaluate our middleware. Compared with the traditional one-size-fits-all paradigm, it reduces user-perceived latency by up to 50% with lightweight runtime overheads (Section 4).

2 Background and Motivation

We empirically demonstrate the limitations of one-size-fits-all service binding. We then explain why existing QoS-aware service selection approaches do not fully address this problem, motivating service polymorphism.

2.1 Current Practice: One-Size-Fits-All Service Binding

When developing a service-oriented application, the app developer has the flexibility to choose from multiple services offered by different vendors, all of which meet the app's functional and non-functional requirements. We define these services as ***Equivalent Services***. For instance, the services *Text Translator* [2], *NLP Translator* [15], and *Lecto Translation* [14] can be considered equivalent when an app requires English-to-Spanish translation and all three meet the translation accuracy specified by the app developer.

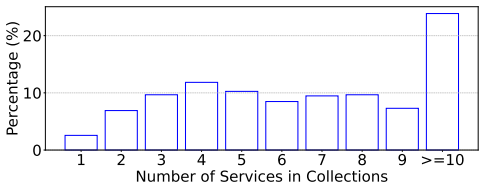


Fig. 2: Distribution of Equivalent Services

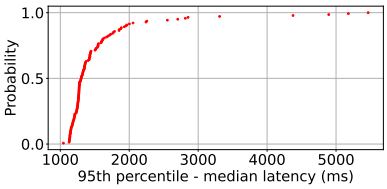


Fig. 3: Latency Inflation

RapidAPI offers collections of web services [3], such as Weather APIs, flight data APIs, Translation APIs, and so on. These collections consist of services

with similar functionalities provided by different vendors that can be used interchangeably in apps with minor adaptions. For instance, the Weather API collection contains over 10 services that can be used to retrieve outdoor temperature and wind information given a zipcode. Fig. 2 provides an overview of the number of services in each collection, with more than 90% of collections featuring 3 or more services, indicating a significant presence of equivalent services.

We then investigated how developers implement service invocations in their apps. We searched GitHub for repositories with RapidAPI service invocations. Since an authentication key, "*x-rapidapi-key*," is necessary to access RapidAPI services, we crawled code snippets using this keyword from GitHub's search interface. This led to the retrieval of 873 unique repositories, limited by GitHub's restrictions. Upon manual inspection of these repositories, we noticed a common trend: *all these apps had hardcoded a single service for each functionality.* Thus, despite the widespread availability of equivalent services, developers still follow a "one-size-fits-all" approach by hardcoding a single service into the application. However, as we show next, web services can experience high QoS fluctuations causing the pre-selected service to perform significantly worse in certain contexts.

2.2 High QoS Fluctuation of Web Services

To investigate service performance characteristics, we randomly selected 150 services from RapidAPI, the world's largest service hub [1], and invoked them from seven global locations: Frankfurt (Germany), Tokyo (Japan), Sydney (Australia), Mumbai (India), Ohio (US), North California (US), and Sao Paulo (Brazil). To avoid service provider-imposed frequency limits, we invoked each service every 45 seconds for three days and recorded the end-to-end latency and HTTP status code of each invocation.

We observed that while many services were highly reliable, with reliability exceeding 99.99%, their latency still varied significantly across invocations. Most invocations experienced low latency, but some showed much higher delays. To quantify this variability, we calculated the absolute inflation between the median latency and the 95th percentile latency for each service. Fig. 3 shows the distribution of latency inflation values across all 150 services. We found that the smallest difference between the median and 95th percentile latencies was around 1000ms, while some services experienced inflation up to 5000ms. Additionally, some services had 95th percentile latencies up to 15 times higher than their median values. These findings show that web service QoS can fluctuate substantially, and such latency spikes can significantly degrade user experience [9].

2.3 Comparing QoS Performance of Equivalent Services

To compare the performance among equivalent services, we selected six sets of equivalent services from RapidAPI based on two criteria: 1) frequent usage in industry and academic research [19], and 2) diverse application purposes. For each set, we chose three equivalent services, prioritizing those with lower subscription costs and higher popularity. Using a method similar to Section 2.2,

we assessed the QoS of these six sets at seven global locations over three days, seeking to answer "*Can one service consistently outperform all equivalent peers*?"

Table 1: Average Latency Performance (in milliseconds) at Different Locations.

Service	Frankfurt	Tokyo	Sydney	Mumbai	Ohio	California	Sao Paulo
Lecto Translation	**389**	**606**	**754**	**640**	395	453	684
NLP Translator	428	758	941	1185	**199**	**367**	**545**
Text Translator	552	948	987	1221	300	498	709

Spatial Comparison: Our analysis revealed that in 4 out of 6 service sets, no single service consistently outperformed the other equivalent alternatives across all locations. For example, Table 1 demonstrates the average latency of three translation services across seven locations. The preselected service, *Lecto*, while having the best overall average latency across all locations, performed poorly in Ohio, North California, and Sao Paulo. In contrast, the alternative service, *NLP Translator*, demonstrated better latency in these locations. This discrepancy could be attributed to differences in server deployments—*Lecto* may lack servers in these regions, while *NLP Translator* has servers deployed there, allowing users in these locations to experience better QoS.

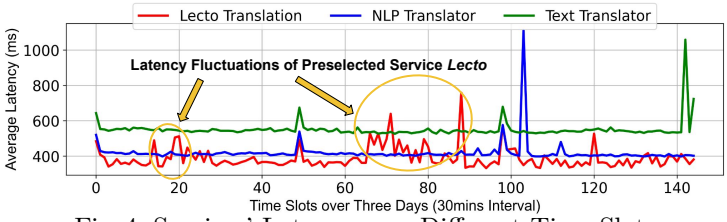


Fig. 4: Services' Latency over Different Time Slots

Temporal Comparison: We also noted that the QoS rankings of equivalent services at one location vary over time. For instance, we calculated the average latencies of translation services at 30-minute intervals in Frankfurt. As shown in Fig. 4, although *Lecto* generally exhibits overall lower average latency among all services, its latency experiences sudden spikes during time slots 15–20 and 65–88. This temporal suboptimality in the performance of *Lecto* could be explained by that: during these time slots, this preselected service likely experienced server overload, while its equivalent peers—*NLP Translator*—did not and performed better, possibly due to different business hours or load-balancing strategies.

2.4 Limitations of Existing Dynamic QoS-Aware Service Selection

The above findings show that equivalent services offer a way to mitigate QoS degradation, but the one-size-fits-all service provisioning paradigm fails to take advantage of this potential. This raises a natural question: *can existing dynamic QoS-aware service selection address the context-varying performance of equivalent services?* Existing studies have extensively adapted service choices according to QoS, but they primarily target different service-selection settings.

Dynamic selection has been widely studied for replicated services. DONAR [18], for example, dynamically directs requests among geographically distributed replicas of the same cloud service according to locality, load, and provider policies.

Client-side approaches similarly select among replicas based on client-observed performance [13]. However, these approaches assume that the service itself has already been selected and optimize which replica serves a request. Our empirical study reveals QoS opportunities across independently provided equivalent services. For example, selecting another replica of *Lecto Translation* cannot exploit the QoS advantage of *NLP Translator* observed in Ohio. QoS-aware service composition further supports dynamic binding among functionally similar services [5,7]. However, these approaches bind abstract tasks to concrete services to optimize the QoS of an overall service composition or workflow. In contrast, the problem revealed by our study occurs at the level of individual end-user invocations: the best service for the same application functionality can differ across user contexts and change over time. A third line of work focuses on QoS prediction. These studies use historical user–service observations to estimate the QoS that a user may experience from candidate services and use the resulting estimates for service recommendation or ranking. The primary objective of these works is to improve QoS estimation accuracy through various techniques, including latent-factor models [22], deep learning [20], and graph-based learning [8]. However, our empirical study reveals a complementary ***client-side service-binding*** problem: how applications can continuously adapt to changing QoS conditions by keeping multiple equivalent services interchangeable and dynamically determining their binding for individual invocations during execution.

3 Polymorphic Service Framework

Service polymorphism introduces a software paradigm for service-oriented applications. Instead of permanently binding an application functionality to a single service during development, a ***polymorphic service*** represents a set of equivalent services that remain interchangeable throughout application execution. Individual service invocations are then dynamically bound to one of these equivalent services according to the QoS observed in the end user's current context. Although our empirical study demonstrates the potential benefits of this paradigm, realizing it in practice is non-trivial. We identify two key challenges:
C1: How can fine-grained QoS be learned without dedicated probing? Service polymorphism requires fresh QoS information for all equivalent services so that we can know the best service for each user at runtime. A straightforward approach is to periodically probe every equivalent service to measure its QoS. However, dedicated probing introduces extra service requests, consumes mobile-device and network resources, and may burden the cloud servers that provides the web services, especially when many users and services are involved. At the same time, reducing probing too aggressively may leave QoS estimates stale and cause changes in service performance to be missed.
***Insight*:** Instead of relying on dedicated probing, QoS can be learned from real requests triggered by users when they invoke a polymorphic service in the app. Each request is eventually served by one equivalent service and naturally produces a QoS observation, such as latency or failure status, which can be reused

for future service selection. However, each polymorphic service invocation still needs to be assigned to one equivalent service. Invoking a service with limited or outdated QoS observations helps refresh its QoS estimate, but may select a suboptimal service for that request; invoking the currently best-estimated service improves immediate user QoS, but may miss changes in other services. This creates an exploration–exploitation trade-off: exploration updates uncertain QoS estimates, while exploitation maximizes current user-perceived QoS. We model this trade-off as a **Multi-Armed Bandit (MAB) problem**, a classical framework for balancing exploration and exploitation, and reshape a UCB-based algorithm customized to service QoS to solve it efficiently, as detailed in Sec. 3.1.
C2: How can individual users avoid excessive exploration? Although QoS can be learned from normal service invocations, an individual user may not invoke a polymorphic service frequently enough to maintain accurate QoS estimates for all equivalent services. As a result, the user may need to explore QoS-uncertain services more often, increasing the chance of selecting suboptimal services and hurting user-perceived QoS. This issue becomes more significant when there are many equivalent services or when service QoS changes frequently.
***Insight*:** Nearby users often experience similar service QoS because they share similar geographic and network contexts. Therefore, QoS observations can be shared among nearby users through crowdsourcing, so that one user's invocation helps others update their QoS estimates. This collaborative sharing reduces the exploration burden on each individual user while improving QoS estimation quality. However, this sharing must be low-overhead: retrieving shared QoS data from a remote cloud or even an edge server several hops away may offset the benefit of selecting a better service. WiFi APs are a better fit because they are only one hop away from users and often have underutilized resources [12].

3.1 Balancing QoS Exploration and Exploitation

MAB problem is an abstraction of the exploration-exploitation process. Here, a gambler faces several slot machines, each with an unknown reward distribution. The gambler's goal is to maximize the average reward over multiple pulls. To achieve this, the gambler will balance exploration (i.e., learning the reward distributions of the different machines) and exploitation (i.e., pulling the machine that offers the highest expected reward). In the context of service polymorphism, each equivalent service is analogous to a "slot machine," and invoking a service (or "pulling an arm") produces a "reward," represented by its QoS. For each user, the goal is to optimize QoS across a sequence of polymorphic service invocations. During exploration, the user invokes services with uncertain or outdated QoS estimates to refresh QoS knowledge. During exploitation, the user invokes the service that shows the best QoS based on past observations. By balancing these two choices, we can maximize long-term average user-perceived QoS.

In addition, QoS is a multi-dimensional vector that includes metrics such as reliability, latency, and availability. A general MAB problem for QoS optimization can be defined as maximizing the multi-dimensional QoS utility. In this work, however, we simplify the problem by focusing exclusively on optimizing

Algorithm 1 Reshaped SW-UCB for Polymorphic Service Invocation

1: **Input:** Sessions S; K services; base window τ_0; period Δt; factor α
2: **Init:** $N_s(i) \leftarrow 0$, $\bar{X}_s(i) \leftarrow 0$, $\forall i \in \{1, \ldots, K\}$; $\tau \leftarrow \tau_0$; $t_{\text{next}} \leftarrow \text{time} + \Delta t$
3: **for** $s = 1$ to S **do**
4: **if** some service i has not been invoked **then**
5: Invoke i, observe L_o, and update $N_1(i) \leftarrow N_1(i) + 1$, $\bar{X}_1(\tau, i) \leftarrow X_1(i)$
6: **else**
7: **for** each service $i \in \{1, \ldots, K\}$ **do**
8: $N_s(\tau, i) \leftarrow \sum_{r=s-\tau+1}^{s} \mathbf{1}_{\{I_r=i\}}$
9: $\bar{X}_s(\tau, i) \leftarrow \frac{1}{N_s(\tau,i)} \sum_{r=s-\tau+1}^{s} X_r(i) \mathbf{1}_{\{I_r=i\}}$
10: $UCB_s(i) \leftarrow \bar{X}_s(\tau, i) - \sqrt{\frac{\xi \log(s \wedge \tau)}{N_s(\tau,i)}}$
11: **end for**
12: Invoke $I_s \leftarrow \arg\min_{i \in \{1,\ldots,K\}} UCB_s(i)$ and observe $X_s(I_s)$
13: **if** current time $\geq t_{\text{next}}$ **then**
14: $\lambda_t \leftarrow \sum_{r \in [t-\Delta t, t]} \mathbf{1}_{\{\text{request at time } r\}}$
15: $\tau \leftarrow \alpha \cdot \tau_0 \cdot \log(1 + \lambda_t)$; $t_{\text{next}} \leftarrow \text{current time} + \Delta t$
16: **end if**
17: **end if**
18: **end for**

service latency, as most services in our testing environment exhibit high reliability. Nonetheless, the algorithm developed here can be applied to address the general MAB problem by normalizing multiple metrics into a single utility value.

Reshaped SW-UCB Algorithm: The MAB problem has been widely studied, Upper Confidence Bound (UCB), in particular, demonstrates strong performance in addressing the exploration-exploitation trade-off. They estimate the average latency for each service (arm) while maintaining a confidence bound that reflects the uncertainty of the estimate (i.e., UCB score). They then select the service with the lowest UCB score, striking a balance between exploiting services with known performance and exploring those with uncertain latencies. Building on the Sliding Window UCB (SW-UCB) [6], which estimates each option's performance using only recent observations within a sliding window to adapt to latency changes, we develop a reshaped version tailored for service invocation.

In Algorithm 1, Lines 4–5 initialize the process by invoking each service at least once to collect initial latency observations. For each subsequent invocation, the algorithm calculates the recent average latency $\bar{X}_s(\tau, i)$ and invocation count $N_s(\tau, i)$ of each service within the sliding window (Lines 8–9). It then computes a selection score for each service (Line 10) and invokes the service with the lowest score (Line 12). The score favors services with lower observed latency while retaining opportunities to refresh services whose QoS remains uncertain. Specifically, we consider two characteristics of service invocation: latency variability and service invocation rates. These considerations are orthogonal to the underlying MAB algorithm and can be incorporated into other MAB approaches as well, although we implement them on top of SW-UCB.

1. Variance-Sensitive Exploration: The first term in Line 10, $\bar{X}_s(\tau, i)$, favors services with lower observed latency, while the second term controls exploration. In vanilla SW-UCB, the exploration term mainly decreases with the number of observations $N_s(\tau, i)$ under a fixed scaling parameter ξ. However, our empirical study shows that web services can exhibit substantially different latency vari-

```
 <NAME> ::= HOSTNAME                  <TTL> ::= NULL
 <TYPE> ::= 400(QoS)              <RDLENGTH> ::= The Length of RDATA
<CLASS> ::= REQUEST/RESPONSE        <RDATA> ::= HASH(URL)
```

Fig. 5: RR Format Definition for DNS-QoS Queries

ability. Thus, two services with the same number of observations may still differ considerably in the stability of their recent latency estimates. We therefore set ξ according to the empirical variance of the observed latency samples, making the selection score sensitive to QoS variability and encouraging additional exploration when recent latency observations are less stable.

2. Request-Rate-Aware Window: Every Δt minutes, the window size τ is dynamically adjusted according to the recent request rate λ_t (Lines 13–16). The scaling factor α controls how strongly the window size responds to changes in request rate. When requests are infrequent, τ shrinks to emphasize fresher observations and reduce the influence of stale QoS samples. When requests are frequent, τ expands to retain more recent samples and improve the stability of latency estimation.

3.2 Collaborative QoS Sharing via WiFi APs

To reduce redundant exploration and improve QoS estimation quality, we enable crowdsourced QoS sharing among nearby users through their connected WiFi AP. The AP collects QoS reports after service invocations and provides shared QoS knowledge to later users for service-selection decisions.

Running the algorithm to decide which equivalent service to invoke requires the user to retrieve shared QoS information from the AP. A straightforward approach is to send a dedicated request to the AP before each polymorphic service invocation. However, this extra request introduces an additional round-trip time between the user and the AP, which can offset the benefit of selecting a better service. Our key observation is that service invocations are typically performed through HTTP requests, which usually require DNS resolution beforehand, and WiFi APs usually serve as local DNS servers. Therefore, we leverage a technique called DNS piggybacking [12]: we embed QoS retrieval into DNS queries, allowing the user to obtain shared QoS data during normal DNS resolution without issuing a separate lookup request, thereby minimizing latency overhead.

The standard DNS packet includes four sections. `Header` stores metadata such as the identification number, while `Question` and `Answer` are used for IP address requests and responses, respectively. `Additional` contains one or more "Resource Record" (RR) items for information related to the query. For example, EDNS [10] uses its own RR type to transfer additional control information. Following this pattern, we use the `Additional` section to define a new RR for transmitting QoS data. We refer to this modified query as a DNS-QoS query, whose format is shown in Fig. 5. The `<Name>` field contains a service's hostname, while the `<TYPE>` field is assigned an unsigned integer value of 400, denoting "QoS" type. The `<CLASS>` field can be either "`REQUEST`" or "`RESPONSE`". For a `REQUEST` RR, the `<RDATA>` field contains the hash code of the service URL,

enabling the AP to look up the associated QoS observations for decision-making. For a `RESPONSE` RR, the `<RDATA>` field contains the QoS statistics required by Algorithm 1, including average latency, invocation count, and latency variance.

3.3 Runtime System Workflow

Building on the above components, we design the Polymorphic Service Framework (PSF), a middleware system that realizes service polymorphism. Its runtime workflow is illustrated in Fig. 6.

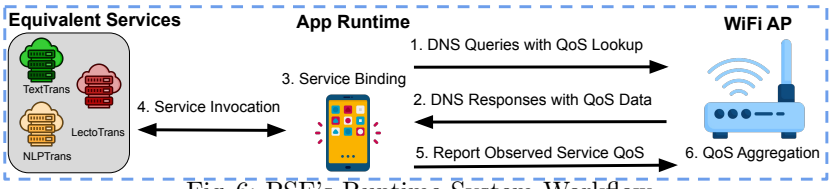


Fig. 6: PSF's Runtime System Workflow

When an app invokes a polymorphic service, the app runtime first identifies the candidate equivalent services configured by the developer. (1) Before issuing the actual service request, the app runtime sends DNS-QoS queries for all equivalent services to its connected WiFi AP. (2) For each equivalent service, the AP returns the IP address together with the shared QoS statistics, including the recent average latency, latency variance, and invocation count. (3) Using these statistics, the app runtime runs the reshaped UCB algorithm locally to choose the equivalent service for the current invocation. (4) The runtime then invokes the selected service and returns the result to the app. (5) After the invocation completes, the runtime reports the observed latency back to the AP. (6) The AP aggregates latency observations from all users to continuously update the shared QoS statistics, including average latency, latency variance, and counts, for future invocations, benefiting all users connected to the same AP.

4 Evaluation

4.1 Testbed Setup

We developed an Android client library with: 1) a native DNS interface using *c-ares*, and 2) a Java library for managing developer inputs, service results, and QoS reporting. The two components communicate via the Java-Native Interface (JNI). The UCB algorithm was configured with base window size $\tau_0 = 30$, scaling factor $\alpha = 0.2$, and window adjustment interval $\Delta t = 30$ minutes. On the AP, we extended *Dnsmasq*, the built-in DNS library of OpenWRT, an open-source Linux-based OS commonly used in WiFi APs. We allocated 5MB of AP memory for storing QoS data. We open-source the implementation code along with all collected service invocation traces in our empirical study.[5]

[5] `https://github.com/fdf234243/Service-Polymorphism`

Evaluated Service Invocation Approaches
I) Pre-selection: Among a set of equivalent services, one service with the minimal average latency was pre-selected (i.e., "one-size-fits-all").
II) Weighted Round-Robin (Weight-RR): We adapt this method from the load balancing domain for service selection, in which the preselected service is assigned a higher weight. We set this weight to 80%, based on the observation that the preselected service is dominant across most times and locations.
III) PSF-AP: This is our solution, utilizing both our reshaped UCB algorithm for service selection and the WiFi AP for QoS sharing. To distinguish it from other PSF-based alternatives, we refer to it as PSF-AP only in the evaluation.
IV) PSF-UCB: This approach has the same workflow as *PSF-AP* but differs from its MAB algorithm, simply utilizing the vanilla UCB for the service selection.
V) PSF-EdgeSharing: This approach uses our reshaped UCB algorithm but replaces WiFi AP-based QoS sharing with edge-server-based QoS aggregation. Before each invocation, the app runtime retrieves shared QoS information from an edge server, runs the algorithm to select a service, and reports the observed QoS back after invocation.
Deployment: We built a testbed that includes a commodity router (GLMT1300 with an MT7621A CPU and 256MB RAM) functioning as the WiFi AP. One Desktop located 7 hops away from the AP serves as the edge server. Five Samsung Galaxy A53 phones are connected to the AP via WiFi. We developed Android apps that invoke the six sets of equivalent services (selected in our empirical study). For each set, we created five versions of an app, each using a different service selection approach (including four baselines and our solution). For each service set, the apps were run simultaneously on all five phones over a 24-hour period. To simulate the variability in service demand across different times of day, we varied the number of service requests per hourly time slot. The number of requests was randomly selected from a predefined list categorized into low (5 requests/h), medium (40 requests/h), and high (80 requests/h) arrival rates. Given a request rate, we utilized the Zipf distribution [11] to determine the time intervals between two consecutive service invocations in each time slot.

4.2 PSF's Performance on Testbed

The performance improvement depends on service characteristics, with two cases. When the pre-selected service does not dominate its equivalents, dynamic switching can reduce user-perceived latency; when it dominates, switching has little effect. Based on pre-selected service performance, "Translation" and "Weather" fall into the first case, while "FaceDet" falls into the second. We use them to compare *PSF-AP* with other approaches. As shown in Fig. 7, *PSF-AP* achieves the largest improvement in the first case while minimizing overhead in the second.
Comparison with Current Practice — Pre-selection:
1) For "Translation," the pre-selected service "Lecto" typically performed well in most locations but exhibited poor latency at the experiment location. *PSF-AP* identified this spatial fluctuation and switched to "NLP Trans," improving average latency by 49.62% and P95 latency by 53.13%.

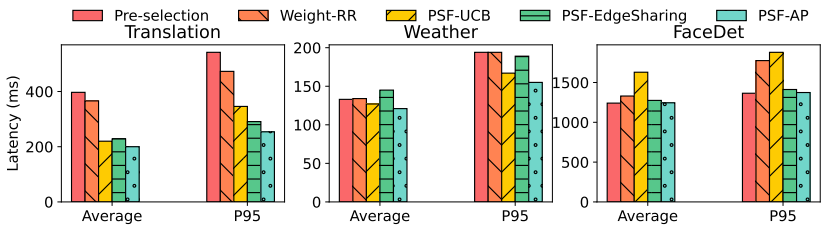


Fig. 7: Performance of Service Invocation Methods

2) For "Weather," the pre-selected service "OpenWeatherMap" generally provided the best latency but suffered from temporal fluctuations. Though its overall performance was similar to "Weatherbito," it performed worse in specific time slots. *PSF-AP* detected these fluctuations and switched to "Weatherbito" when necessary, improving average latency by 9.02% and P95 latency by 20.10%.
3) For "FaceDet," the pre-selected service "Microsoft" provided best latency in all time slots at the experiment location. As the pre-selected service dominated, *PSF-AP* maintained comparable latency to *Pre-selection* by efficiently balancing exploration and exploitation while keeping communication overhead low.
Comparison with Alternative Implementations: We compared the performance of *PSF-AP* with that of three alternatives that dynamically invoke service. We observed that: 1) Alternative approaches improved service latency only when the pre-selected service exhibited significantly worse performance than its equivalents (see "Translation"). In such cases, these methods occasionally invoked other equivalent services and, due to the large latency gap, achieved better overall latency across all invocation sessions. 2) Only *PSF-AP* consistently improved performance without downgrading the results compared to *Pre-selection*.

In particular, *PSF-UCB* struggled to balance QoS exploration and exploitation, frequently invoking suboptimal services, which led to inferior QoS. Similarly, *PSF-EdgeSharing*, while using the reshaped UCB algorithm, suffered from high latency overhead because clients had to consult a distant edge server to decide which service to invoke. This additional latency offsets the benefits of invoking the optimal service, at times resulting in worse latency than *Pre-selection* (see "Weather" and "FaceDet"). All these observations highlight that only *PSF-AP* can be practically adopted to implement service polymorphism.

4.3 System Overhead and Scalability

PSF introduces additional processing at the AP through DNS-QoS queries and QoS aggregation. We therefore evaluate its runtime overhead and scalability under increasing workloads. We use Python to emulate different numbers of users connected to the AP. Each emulated user periodically issues DNS-QoS queries and reports service latency observations. The reported latencies are sampled from the QoS traces collected in our empirical study, while the request arrival rates follow the same workload settings used in the previous experiments. For each workload, we measure three metrics: (1) the additional latency introduced by DNS-QoS compared with regular DNS queries, (2) the increase in AP CPU utilization relative to running without PSF, and (3) the increase in AP memory consumption relative to running without PSF.

Table 2: PSF overhead under different workloads.

# Users	DNS-QoS Overhead (ms)	CPU Increase (%)	Memory Increase (MB)
20	0.03 ± 0.01	4.1 ± 0.5	11.0 ± 0.4
50	0.04 ± 0.02	7.8 ± 0.7	11.8 ± 0.5
100	0.06 ± 0.02	12.4 ± 1.0	12.9 ± 0.6
200	0.09 ± 0.03	18.7 ± 1.4	14.7 ± 0.8

Table 2 shows the resulting overhead. As the workload increases, both CPU and memory consumption grow gradually due to the additional DNS-QoS processing and QoS aggregation performed by the AP. Even under the largest workload of 200 users, CPU utilization increases by less than 20% and memory consumption by less than 15 MB compared with running without PSF. More importantly, the additional latency introduced by DNS-QoS remains extremely small across all workloads. Even at 200 users, the average overhead remains below 0.1 ms compared with regular DNS queries. This is because PSF piggybacks compact QoS statistics onto DNS responses and performs only lightweight aggregation at the AP. Overall, these results demonstrate that PSF remains lightweight and scalable under increasing workloads. A commodity WiFi AP can efficiently support collaborative QoS sharing for a large number of users while introducing negligible latency overhead to service invocations.

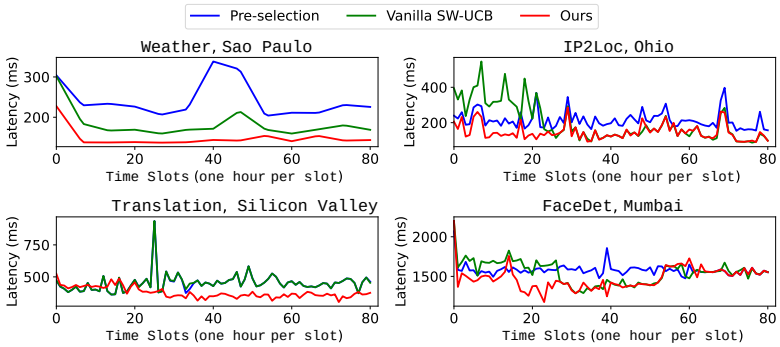


Fig. 8: Exploration and Exploitation Processes of UCB Algorithms

4.4 World-wide Performance Improvement

To evaluate the benefits of PSF across different locations, we compared our reshaped algorithm with vanilla SW-UCB using the QoS traces collected in our empirical study. Both algorithms were implemented in Python and evaluated on each service set. To emulate varying request rates, we partitioned each service trace into hourly slots and randomly selected a request rate from the workload settings used in our testbed. Based on the selected rate, invocation indices were randomly sampled within the slot. Since the original traces were collected at 45-second intervals (80 requests/hour), the sampled indices were also used to determine the window-adjustment timing in the reshaped algorithm.

Because the pre-selected services in the "Flight" and "Hotel" sets consistently outperformed their equivalents and therefore offered no optimization opportunity, we evaluated only the remaining four service sets: "Translation," "Weather,"

"FaceDet," and "IP2Loc." To account for randomness, each experiment was repeated 10 times and the average results are reported.

Compared to vanilla SW-UCB: We use results from each service set to illustrate the exploration and exploitation process of the two algorithms. As shown in Fig. 8, our algorithm achieves lower latency than the vanilla version across all four sets by exploring fewer suboptimal services, due to its customization for service invocation. In particular, for sets such as "IP2Loc, Ohio" and "FaceDet, Mumbai," where equivalent services show high latency variance at the beginning, our algorithm better avoids suboptimal selections by incorporating variance into the exploration factor.

Table 3: Optimal Service Selection Rate and Latency Improvement

	Weather		**IP2Loc**		**Translation**		**FaceDet**	
Algorithms	Opt Rate	Lat Imp	Opt Rate	Lat Imp	Opt Rate	Lat Imp	Opt Rate	Lat Imp
Vanilla SW-UCB	34%	17%	79%	-55%	31%	11%	61%	3%
Ours	96%	35%	95%	30%	76%	25%	72%	7%

We also present the average latency improvement ratio compared to the pre-selected service for the four service sets, focusing on traces from locations where the latency performance of the pre-selected service was not dominant. Along with the average rate at which each algorithm selected the optimal service, these two metrics are shown in Table 3. Our observations are as follows: 1) Our algorithm consistently achieved higher latency improvements than the vanilla SW-UCB, with an average improvement of 24.25% across all four sets over *Pre-selection*. 2) Our algorithm had a higher optimal service selection rate than the vanilla version, which explains its superior latency performance.

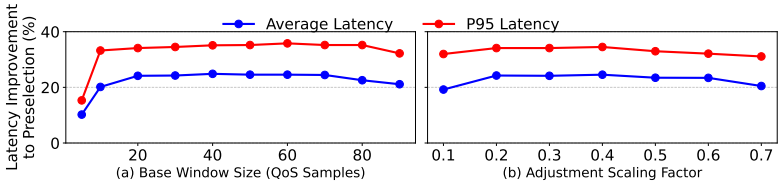


Fig. 9: Impact of Parameters on Algorithm's Performance

Robustness Analysis: We then investigate how different parameter values influence the algorithm's latency improvement compared to *Pre-selection*. As shown in Fig. 9, we observe that increasing the window size (i.e., including more recent QoS samples) initially leads to a significant performance boost, as larger sample sizes (e.g., increasing from 5 to 10 samples) improve the accuracy of latency estimates. However, performance begins to degrade when the window becomes too large, as it incorporates stale QoS samples. In contrast, the algorithm is less sensitive to changes in the window adjustment scaling factor. Modifying these parameters has a less pronounced impact on performance. As it belongs to the dynamic window module, this result also shows that latency variance contributes more to the final performance than the dynamic window.

5 Conclusion and Future Work

This paper introduces service polymorphism, a software paradigm that addresses QoS fluctuation by dynamically binding service invocations to functionally equivalent services on a per-request basis. Rather than relying on a fixed service instance, a polymorphic service selects an equivalent implementation at runtime according to the end user's current QoS context. Experimental results show that PSF reduces user-perceived latency by up to 50% with negligible runtime overhead. As future work, we plan to investigate robustness against malicious or inaccurate QoS reports. Existing trust and reputation mechanisms from the crowdsourcing literature could be incorporated into PSF to improve the reliability of collaboratively shared QoS information. Another promising direction is extending the current AP-local QoS sharing mechanism to a hierarchical architecture spanning multiple APs and edge nodes, allowing service polymorphism to leverage larger-scale QoS knowledge while maintaining scalability.

Acknowledgement

This research is supported by NSF through grant 2104337.

References

1. Rapidapi. https://rapidapi.com/, accessed: 2025-03-30
2. Texttranslator, rapidapi.com/dickyagustin/api/text-translator2
3. All api collections on rapidapi. https://rapidapi.com/collections (2024)
4. Baravkar, S., Gaikwad, P., Li, Z., Tilevich, E., Cheng, L., Song, Z.: Towards a comprehensive understanding of web service integration: a large-scale empirical study from the developers' perspective. EMSE **30**(5), 144 (2025)
5. Cardellini, V., Casalicchio, E., Grassi, V., Presti, F.L., Mirandola, R.: QoS-driven runtime adaptation of service oriented architectures. In: the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering. pp. 131–140 (2009)
6. Garivier, A., Moulines, E.: On upper-confidence bound policies for non-stationary bandit problems. arXiv preprint arXiv:0805.3415 (2008)
7. Grassi, V., Marzolla, M., Mirandola, R.: QoS-aware fully decentralized service assembly. In: Proceedings of the 8th SEAMS. pp. 53–62. IEEE (2013)
8. Guo, C., Yin, Y., Wu, Q., Qiu, W., Zhang, M., Zhou, X.: QoS-aware diversified service selection. IEEE Transactions on Services Computing **16**(3), 2084–2097 (2023)
9. Haque, M.E., Eom, Y.H., He, Y., Elnikety, S., Bianchini, R., McKinley, K.S.: Few-to-many: Incremental parallelism for reducing tail latency in interactive services. ACM Sigplan Notices **50**(4), 161–175 (2015)
10. IETF: Extension mechanisms for DNS (EDNS(0)). https://datatracker.ietf.org/doc/html/rfc6891, accessed: 2024-05-29
11. Kihl, M., Larsson, R., Unnervik, N., Haberkamm, J., Arvidsson, A., Aurelius, A.: Analysis of facebook content demand patterns. In: SaCoNeT. pp. 1–6. IEEE (2014)
12. Li, Z., Shrestha, S., Song, Z., Tilevich, E.: Edge cache on wifi access points: Millisecond-level app latency almost for free. In: ICDCS. pp. 601–612. IEEE (2024)

13. Mendonça, N.C., Silva, J.A.F., Anido, R.O.: Client-side selection of replicated web services: An empirical assessment. Journal of Systems and Software **81**(8), 1346–1363 (2008). https://doi.org/10.1016/j.jss.2007.11.002
14. RapidAPI: Lecto translation (April 2024), `https://shorturl.at/ekp6h`
15. RapidAPI: Nlp translation (April 2024), `https://shorturl.at/3bv1f`
16. RapidAPI: Consume any api using model context protocol (mcp). `https://docs.rapidapi.com/docs/consume-apis-using-ai` (2026)
17. Segura, S., Parejo, J.A., Troya, J., Ruiz-Cortés, A.: Metamorphic testing of restful web apis. In: IEEE/ACM ICSE. pp. 882–882 (2018)
18. Wendell, P., Jiang, J.W., Freedman, M.J., Rexford, J.: DONAR: Decentralized server selection for cloud services. In: ACM SIGCOMM'2010. pp. 231–242 (2010)
19. Wu, Z., Madhyastha, H.: Understanding the latency benefits of multi-cloud web-service deployments. SIGCOMM **43**, 13–20 (04 2013)
20. Xu, Z., Zhang, J., et al.: Nfmf: Neural fusion matrix factorisation for qos prediction in service selection. IET Software **15**(2), 101–114 (2021)
21. Zhang, H., Huang, S., Xu, M., Guo, D., Wang, X., Wang, X., Leung, V.C., Wang, W.: Large-scale measurements and optimizations on latency in edge clouds. IEEE Transactions on Cloud Computing (2024)
22. Zheng, Z., Ma, H., Lyu, M.R., King, I.: Collaborative web service QoS prediction via neighborhood integrated matrix factorization. IEEE Transactions on Services Computing **6**(3), 289–299 (2013). https://doi.org/10.1109/TSC.2011.59